

SKILLWEEK5

TASK1: To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

```
jhansi_priya@Jhansi-Priya: ~/  × + v - □ >
jhansi_priya@Jhansi-Priya:~$ mkdir Skill5
jhansi_priya@Jhansi-Priya:~$ cd Skill5
jhansi_priya@Jhansi-Priya:~/Skill5$ nano task1.c
jhansi_priya@Jhansi-Priya:~/Skill5$ gcc task1.c -o task1
```

```
jhansi_priya@Jhansi-Priya: ~/  × + v - □ ×
jhansi_priya@Jhansi-Priya:~$ cd Skill5
jhansi_priya@Jhansi-Priya:~/Skill5$ nano task1.c
jhansi_priya@Jhansi-Priya:~/Skill5$ gcc task1.c -o task1
jhansi_priya@Jhansi-Priya:~/Skill5$ ./task1
Current directory: /home/jhansi_priya/Skill5
Changing to /tmp...
New directory: /tmp
Previous directory: /home/jhansi_priya/Skill5
jhansi_priya@Jhansi-Priya:~/Skill5$ |
```

```
GNU nano 7.2 task1.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <limits.h>

int main() {
    char current[PATH_MAX];
    char previous[PATH_MAX];

    getcwd(current, sizeof(current));
    strcpy(previous, current);

    printf("Current directory: %s\n", current);

    printf("Changing to /tmp...\n");

    if (chdir("/tmp") == 0) {
        strcpy(previous, current);

        getcwd(current, sizeof(current));

        printf("New directory: %s\n", current);
        printf("Previous directory: %s\n", previous);
    } else {
        perror("cd failed");
    }

    return 0;
}

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

TASK2: To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
jhansi_priya@Jhansi-Priya: ~/  ×  +  ∨  
GNU nano 7.2  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <unistd.h>  
  
void pwd_command() {  
    char path[1024];  
  
    if (getcwd(path, sizeof(path)) != NULL)  
        printf("%s\n", path);  
    else  
        perror("pwd");  
}  
  
void cd_command(char *path) {  
    if (path == NULL) {  
        printf("cd: missing argument\n");  
        return;  
    }  
  
    if (chdir(path) != 0)  
        perror("cd");  
}  
  
void exit_command() {  
    printf("Exiting shell...\n");  
    exit(0);  
}  
  
int main() {  
    char input[100];  
    char *command;
```

```
jhansi_priya@Jhansi-Priya: ~/ × + ✓
GNU nano 7.2
char *command;
char *argument;

while (1) {
    printf("myshell> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    command = strtok(input, " ");
    argument = strtok(NULL, " ");

    if (command == NULL)
        continue;

    if (strcmp(command, "pwd") == 0) {
        pwd_command();
    }
    else if (strcmp(command, "cd") == 0) {
        cd_command(argument);
    }
    else if (strcmp(command, "exit") == 0) {
        exit_command();
    }
    else {
        printf("myshell: invalid command\n");
    }
}

return 0;
}
```

```
jhansi_priya@Jhansi-Priya:~/Skill5$ nano task2.c
jhansi_priya@Jhansi-Priya:~/Skill5$ gcc task2.c
jhansi_priya@Jhansi-Priya:~/Skill5$ gcc task2.c -o myshell
jhansi_priya@Jhansi-Priya:~/Skill5$ ./myshell
myshell> pwd
/home/jhansi_priya/Skill5
myshell> cd /tmp
myshell> pwd
/tmp
myshell> hello
myshell: invalid command
myshell> exit
Exiting shell...
jhansi_priya@Jhansi-Priya:~/Skill5$ |
```

TASK3: To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

```
jhansi_priya@Jhansi-Priya: ~/ × + ▼
GNU nano 7.2 task3.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>

void show_current_directory() {
    char path[PATH_MAX];

    if (getcwd(path, sizeof(path)) != NULL) {
        printf("Current Directory: %s\n", path);
    } else {
        perror("getcwd");
    }
}

int main() {

    printf("Testing pwd command:\n");
    show_current_directory();

    printf("\nTesting exit request...\n");

    printf("Resources cleaned successfully.\n");
    printf("Program exiting...\n");

    return 0;
}
```

```
jhansi_priya@Jhansi-Priya:~/Skill5$ nano task3.c
jhansi_priya@Jhansi-Priya:~/Skill5$ gcc task3.c -o task3
jhansi_priya@Jhansi-Priya:~/Skill5$ ./task3
Testing pwd command:
Current Directory: /home/jhansi_priya/Skill5

Testing exit request...
Resources cleaned successfully.
Program exiting...
jhansi_priya@Jhansi-Priya:~/Skill5$
```

TASK4: To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```
jhansi_priya@Jhansi-Priya: ~/ × + ⌵
GNU nano 7.2 task4.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {

    char variable[] = "STUDENT";
    char value[] = "Jhansi";

    if (setenv(variable, value, 1) == 0) {
        printf("Variable exported successfully.\n");
    } else {
        perror("setenv");
        return 1;
    }

    printf("STUDENT = %s\n", getenv("STUDENT"));

    return 0;
}
```

```
Program exiting...
jhansi_priya@Jhansi-Priya:~/Skill5$ nano task4.c
jhansi_priya@Jhansi-Priya:~/Skill5$ gcc task4.c -o task4
jhansi_priya@Jhansi-Priya:~/Skill5$ ./task4
Variable exported successfully.
STUDENT = Jhansi
```

```
jhansi_priya@Jhansi-Priya: ~/ × + ⌵ — □ ×
GNU nano 7.2 child.c *
#include <stdio.h>
#include <stdlib.h>

int main() {

    char *value = getenv("STUDENT");

    if (value != NULL)
        printf("Child process received STUDENT = %s\n", value);
    else
        printf("STUDENT variable not found\n");

    return 0;
}
```

Program exiting...

```
jhansi_priya@Jhansi-Priya:~/Skill5$ nano task4.c
jhansi_priya@Jhansi-Priya:~/Skill5$ gcc task4.c -o task4
jhansi_priya@Jhansi-Priya:~/Skill5$ ./task4
Variable exported successfully.
STUDENT = Jhansi
```

```
GNU nano 7.2 parent.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    setenv("STUDENT", "Jhansi", 1);
    printf("Parent: STUDENT = %s\n", getenv("STUDENT"));
    pid_t pid = fork();
    if (pid == 0) {
        execl("./child", "child", NULL);
        perror("execl");
        exit(1);
    }
    wait(NULL);
    return 0;
}
```

```
jhansi_priya@Jhansi-Priya:~/Skill5$ ./parent.c
-bash: ./child.c: Permission denied
jhansi_priya@Jhansi-Priya:~/Skill5$ nano parent.c
jhansi_priya@Jhansi-Priya:~/Skill5$ gcc parent.c -o parent
jhansi_priya@Jhansi-Priya:~/Skill5$ ./parent
Parent: STUDENT = Jhansi
Child process received STUDENT = Jhansi
jhansi_priya@Jhansi-Priya:~/Skill5$ |
```

